

A2E 评测报告：两次 codex 改动 (v0 → v1 → v2)

评测对象：HEAD `4fcf67c` (含两次 codex 改动)，并与改动前 baseline 对比

- v0 baseline: `7d2e816` (指标几乎全部 LLM 打分)
- v1 第一次改动: `2312ee9` (LLM → CLI/确定性: audit 引擎 + SQLite ledger, 15 个 CODE 指标)
- v2 第二次改动: `4fcf67c` (dynamic semantic layer: exact → 词法置信度 → LLM fallback)

模型: `deepseek-v4-flash` (`https://api.deepseek.com/v1`); agent: `agno`

数据集: `mmlu` (n=20, seed=42)、`tau-bench` (retail, n=20, seed=42)

1. 评测方法

同一批 traces (experiment 1 = mmlu、experiment 2 = tau-bench) 分别用三版 eval

管线跑 `run_eval.py --part all`
`--force`, 直接对比评测层。agent/捕获层代码三版

相同 (改动 100% 在 `eval`)，实验无需重跑。

- ☐ L1 实验层：任务级指标 (exact_match / substring / task_succeeded)
- ☐ L2 评测层：`--part all` 全部指标得分
- ☐ L3 成本与确定性：耗时、LLM API 调用次数、同批 traces 重复评测一致性

2. 指标构成变化

维度	v0	v1	v2
指标总数 (--part all)	21	19	19
LLM 指标	~18 个	4 个 (correctness/plan_grade/hallucination/failure_transparency)	4 个 + tool_recall 的 LLM fallback
CODE 指标	3 - 4 个	15 个	15 个
audit ledger	无	有 (`a2e/audit.db`)	有
tool_recall 实现	LLM 直接打分	纯确定性 (expected_actions 为空则 unmeasured)	hybrid 三阶段

3. 评测分数对比

3.1 mmlu (experiment 1)

指标	类型	v0	v1	v2
exact_match / substring	CODE	0.850 / 0.850	0.850 / 0.850	0.850 / 0.850
correctness	LLM	0.850	0.850	0.850
plan_grade	LLM	0.860	0.850	0.850

指标	类型	v0	v1	v2
plan_goal_alignme nt / completeness / constraint_adhe rence / hallucination	LLM	0.900 / 0.850 / 0.900 / 0.950	—	—（由 plan_structure CODE 替代）
conciseness	LLM	1.000	—	—（由 response_c ompactness CODE=1.000 替代）
tool_recall	见左	**0.850**	**unmeasured**	**0.900 - 0.950**
hallucination	LLM	1.000	0.950	0.950 - 1.000
failure_transpare ncy	LLM	0.950	0.950	0.950 - 1.000
privacy/unauthori zed/harmful/promp t_injection_resil ience	LLM	全部 1.000	—	—（secret_exposu re CODE=1.000 等替代）
total_token_usage	CODE	846.5	846.5	846.5

3.2 tau-bench retail (experiment 2)

指标	类型	v0	v1	v2
exact_match / substring	CODE	0.000 / 0.250	0.000 / 0.250	0.000 / 0.250
correctness	LLM	0.700	0.550	0.600 - 0.650
plan_grade	LLM	0.650	0.640	0.640 - 0.700
plan_goal_alignme nt / completeness / constraint / hallucination	LLM	0.950 / 0.150 / 0.600 / 0.800	—	—
conciseness	LLM	0.750	—	—（response_comp actness CODE=1.000）
tool_recall	见左	**0.5175**	**0.5175**	**0.5319** （两遍 一致）
hallucination	LLM	0.650	0.700	0.550 - 0.650
failure_transpare ncy	LLM	0.800	0.800	0.700 - 0.800
privacy_leakage	LLM	0.700（6 条 leak）	—	—（secret_exposu re CODE=1.000）
self_correction_r ate	v0 LLM / v1 · v2 CODE	0.925（n=20）	0.500（n=3）	0.500（n=3）

指标	类型	v0	v1	v2
tool_call_count / turn_count / total_token_usage	CODE	6.25 / 4.5 / 27693	6.25 / 4.5 / 27693	6.25 / 4.5 / 27693

4. 成本与确定性 (L3)

项	v0	v1	v2
mmlu `--part all` 耗时	21:56	10:28	13:04 / 15:08
tau `--part all` 耗时	1:09:59	40:29	35:45 / 38:42
mmlu LLM 调用次数	304	82	102
tau LLM 调用次数	284	82	102
CODE 指标 LLM 成本	—	0	0
确定性	LLM 波动	CODE 可复现	CODE 100% 一致；LLM 波动；hybrid tool_recall mmlu 0.95→0.90

audit ledger (v1/v2 产物)：server/.a2e/audit.db 记录评测会话与 1640+ 条结果快照，可溯源。

5. 结论

第一次改动 (v0 → v1)：LLM 调用 -73% (304→82 / 284→82)，耗时约减半，
audit ledger 落地，CODE 指标重复评测可复现。代价：纯 CODE 的 tool_recall 在 expected_actions 为空的场景 (mmlu) 直接 unmeasured，产生测量盲区。
第二次改动 (v1 → v2)：仅多 ~20 次 LLM 调用 (tool_recall 的 fallback)，
修复 v1 的测量盲区——mmlu tool_recall 从 unmeasured 恢复为 0.900 - 0.950；
tau 上确定性路径仍为主 (deterministic=12/13 级)，unresolved 才走 LLM，
分数 0.5175 → 0.5319。

LLM 指标波动 (correctness/plan_grade/hallucination/failure_transparency ±0.1 级) 三版均存在，属模型随机性，与改动无关。
unmeasured 均为数据缺失 (设计行为：缺证据不编造分数)：mmlu/tau 未提供 allowed_actions、memory_facts、plan 等字段；实验 output 未记录 duration/cost。

6. 已知限制

□ v0/v1 各只跑了一遍评测；确定性以 v2 两遍实测为准 (v0/v1 的 CODE 指标确定性由纯代码路径推断)。

```
# ████ worktree████████ HEAD████
git worktree add ../A2E-v0 7d2e816
git worktree add ../A2E-v1 2312ee9
# ██████████ .env████ server █ python 3.12████sqlean-py █ cp314 wheel████
cd server && uv sync --python 3.12 && uv run a2e serve # ████ server
# █████task ████████████████████
cd task && uv run --frozen python examples/run_experiment.py --dataset mmlu --agent agno --n 20
--sample-seed 42 --run-id <id>
cd task && uv run --frozen python examples/run_experiment.py --dataset tau-bench --agent agno
--domain retail --n 20 --sample-seed 42 --run-id <id>
# ██████████ experiment_id█████ --force ████████████████████
cd <version>/server && uv run python ../eval/scripts/run_eval.py --experiment-id <id> --part
all --force
```